

Lecture on Minimax Algorithm

Course: Artificial Intelligence

Instructor: Syed Hamed Raza

Overview of the Lecture:

In this lecture, we will explore the **Minimax Algorithm**, a fundamental algorithm used in **game theory** and artificial intelligence, particularly in the context of **adversarial search**. The Minimax algorithm is used to make optimal decisions in two-player, zero-sum games, where one player's gain is the other player's loss. By the end of the lecture, you will understand how the Minimax algorithm works, its applications, and how it can be optimized using techniques such as **Alpha-Beta Pruning**.

Agenda:

1. Introduction to Adversarial Search
 2. Minimax Algorithm Overview
 3. Steps Involved in the Minimax Algorithm
 4. Example of Minimax in Action
 5. Alpha-Beta Pruning (Optimization)
 6. Classroom Discussion and Problem Solving
-

1. Introduction to Adversarial Search

What is Adversarial Search?

- **Adversarial search** involves situations where there are multiple agents (players) competing against each other, and each agent's actions affect the others. A common example is **game-playing** where two players make alternate moves, and each player seeks to maximize their own outcome while minimizing the opponent's outcome.
- These types of problems are typically **zero-sum games**, where one player's win is the other player's loss.

Key Characteristics of Adversarial Search:

- **Two Players:** There are two players involved, often referred to as the **MAX player** and the **MIN player**.
- **Zero-Sum:** The total payoff to all players is constant (one player's win is the other player's loss).
- **Perfect Information:** Each player knows the entire state of the game.

The **Minimax Algorithm** is the most common algorithm used for decision-making in these types of games.

2. Minimax Algorithm Overview

The **Minimax Algorithm** is a recursive strategy for selecting the best move for a player assuming the opponent is also playing optimally. The algorithm builds a **game tree**, where each node represents a game state, and each edge represents a possible move. The algorithm evaluates the nodes from the leaves (end states) back to the root (current state), selecting the optimal move based on the assumption that both players are playing optimally.

Objective:

- **MAX player:** The player aims to **maximize** the score (gain).
 - **MIN player:** The opponent aims to **minimize** the score (the MAX player's gain).
-

3. Steps Involved in the Minimax Algorithm

Step 1: Build the Game Tree

- The first step in the Minimax algorithm is to construct a **game tree**, which represents all possible moves from the current state to the terminal states of the game.
- Each node represents a possible game state, and each edge represents a player's move.

Step 2: Assign Values to Terminal States

- Terminal states are the final outcomes of the game (e.g., win, loss, or draw). These states are assigned numeric values to represent their utility:
 - **Win:** Positive value (e.g., +1).
 - **Loss:** Negative value (e.g., -1).
 - **Draw:** Neutral value (e.g., 0).

Step 3: Propagate the Values Back to the Root

- After assigning values to the terminal nodes (leaf nodes), the algorithm moves back toward the root of the tree:
 - At **MAX** nodes (where the MAX player is to move), select the **maximum value** from the child nodes.
 - At **MIN** nodes (where the MIN player is to move), select the **minimum value** from the child nodes.

Step 4: Choose the Best Move

- Once the tree has been evaluated, the root node will have a value that represents the optimal outcome for the MAX player. The MAX player will then choose the move corresponding to the child node with the highest value.

Minimax Pseudocode:

```
function minimax(node, depth, maximizingPlayer):  
    if depth == 0 or node is a terminal node:  
        return evaluate(node)
```

```

if maximizingPlayer:
    maxEval = -∞
    for each child of node:
        eval = minimax(child, depth - 1, false)
        maxEval = max(maxEval, eval)
    return maxEval
else:
    minEval = ∞
    for each child of node:
        eval = minimax(child, depth - 1, true)
        minEval = min(minEval, eval)
    return minEval

```

4. Example of Minimax in Action

Consider a simple two-player game with the following game tree (values represent terminal states):



- The **MAX player** starts at the root.
- The **MIN player** moves to the first level, minimizing the possible outcomes.
- The values at the leaves are 3, 5, 2, and 9.
- For the first MIN node, the minimum value between 3 and 5 is **3**.
- For the second MIN node, the minimum value between 2 and 9 is **2**.
- For the MAX player, the maximum value between the two MIN nodes is **3**.

Thus, the optimal move for the MAX player is to move to the left, resulting in a value of **3**.

5. Alpha-Beta Pruning (Optimization)

Although the **Minimax Algorithm** is effective, it can be inefficient because it evaluates many nodes that do not need to be explored. **Alpha-Beta Pruning** is a technique to **optimize** the Minimax algorithm by **pruning** branches of the game tree that cannot possibly influence the final decision.

How Alpha-Beta Pruning Works:

- Alpha-beta pruning keeps track of two values:
 - **Alpha**: The best value that the maximizer can guarantee so far.
 - **Beta**: The best value that the minimizer can guarantee so far.

- **Pruning** occurs when a branch cannot influence the outcome (i.e., if the value of a node is worse than the alpha or beta value).

By pruning subtrees where further exploration is unnecessary, alpha-beta pruning can reduce the number of nodes evaluated, making the algorithm faster.

6. Classroom Discussion and Problem Solving

- **Open Discussion:** How does Minimax work in games like Tic-Tac-Toe or Chess? What challenges arise when applying Minimax to more complex games?
 - **Hands-on Activity:** Implement the **Minimax Algorithm** for a simple two-player game (e.g., Tic-Tac-Toe) and compare the performance with and without **Alpha-Beta Pruning**.
-

Conclusion:

- **Summary:** In this lecture, we have explored the **Minimax Algorithm** used for decision-making in adversarial games. We discussed how it works by constructing a game tree and selecting the optimal moves for the players involved. We also introduced **Alpha-Beta Pruning**, a technique that optimizes the Minimax algorithm by pruning unnecessary branches in the search tree.
- **Next Steps:** In the next lecture, we will delve into more complex search techniques, including **Monte Carlo Tree Search (MCTS)**, and explore their applications in AI-based game playing.